SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I Oviya M / 192321117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***oviya m***

SIMATS ENGINEERING

CO2_AT3 Error Analysis Task - Answers

COURSE NAME: Software Testing **COURSE CODE:** CSA37

Name: Oviya M **Register No:** 192321117

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

1. Introduction and Approach

This document answers the CO2_AT3 Error Analysis Task for Software Testing (CSA37). Every question describes a flawed testing practice. For each one the answer follows the same five steps used in the rubric: (1) identify all errors, (2) analyse the root causes, (3) propose a corrected solution (test cases, partitions, CFG or paths), (4) justify the correction using testing theory, and (5) conclude with preventive measures.

2. Key Concepts Used

Concept	Definition	Where used
Equivalence Class Partitioning (ECP)	Divide the input domain into disjoint classes whose members are expected to behave identically; test one value per class.	Q1, Q5
Boundary Value Analysis (BVA)	Test at and next to the edges of each class (min-1, min, min+1, max-1, max, max+1) because defects cluster there.	Q1, Q2, Q5
Decision Table Testing	Table of condition combinations (2^n rules) with the action for each rule.	Q7
Control Flow Graph (CFG)	Directed graph: nodes = statements/blocks, edges = possible flow; decisions have two outgoing edges.	Q3, Q10
Cyclomatic complexity V(G)	V(G) = E - N + 2 = P + 1 (P = predicate nodes). Gives the number of independent paths and the minimum tests for basis path coverage.	Q3, Q10
Statement / Branch / Condition / MC/DC / Path coverage	White-box adequacy criteria of increasing strength.	Q4, Q6, Q8
Loop testing	Skip, one, two, typical, max-1, max, max+1 iterations; nested loops innermost first.	Q8
State transition / Error guessing	Sequence-based testing of states and events; experience-based supplement.	Q9

3. Strength of Coverage Criteria

Criterion	What must be executed	Weakness
Statement	Every statement once	Ignores false outcomes of decisions
Branch / Decision	Every decision true and false	Ignores individual conditions
Condition	Every atomic condition true and false	May not execute both decision outcomes
MC/DC	Each condition shown to independently change the decision (n+1 tests)	Needs careful test design
Basis path	V(G) independent paths	Does not test all loop counts
Complete path	All paths	Impossible with loops

<table>
<tr><th>Sl. No</th><th>Answers</th><th>Bloom's Level</th></tr>
<tr><td>1</td><td>
Concept recap: ECP assumes every value in a class is processed the same way. If a class contains both valid and invalid values this assumption is false and the technique loses its meaning.
Assumed system: Quantity field of the shopping cart accepts integers 1 to 100 (valid).
Errors identified
• Valid and invalid values mixed in one class (e.g. one class "-5 to 150" or "0 to 100"), so one test cannot represent the whole class.
• No separate invalid classes (below minimum, above maximum, non-numeric, decimal, empty).
• Boundary Value Analysis not applied: 0, 1, 100, 101 never tested.
• Only one test per class, so off-by-one defects (< vs <=) go unseen.
Root causes
• Classes were not required to be disjoint and homogeneous (every member must behave the same way).
• Partitioning was done from the UI/data type and not from the requirement rule.
• BVA was treated as optional; no review checklist or traceability to the specification.
Corrected test cases: Valid class and invalid classes are separated, and boundaries are added:
<table>
<tr><th>ID</th><th>Class / Boundary</th><th>Input</th><th>Expected</th></tr>
<tr><td>EC1</td><td>Valid 1-100</td><td>50</td><td>Accepted</td></tr>
<tr><td>EC2</td><td>Invalid < 1</td><td>-5</td><td>Error message</td></tr>
<tr><td>EC3</td><td>Invalid > 100</td><td>150</td><td>Error message</td></tr>
<tr><td>EC4</td><td>Invalid non-numeric</td><td>abc</td><td>Error message</td></tr>
<tr><td>EC5</td><td>Invalid decimal</td><td>2.5</td><td>Error message</td></tr>
<tr><td>EC6</td><td>Invalid empty</td><td>(blank)</td><td>Error message</td></tr>
<tr><td>B1-B6</td><td>Boundaries</td><td>0, 1, 2, 99, 100, 101</td><td>Reject, Accept, Accept, Accept, Accept, Reject</td></tr>
</table>
Justification: Homogeneous classes mean any one failing test proves the whole class is faulty. BVA targets the boundaries where most conditional (<, <=) defects live. The set stays small (about 13 tests) yet covers every behaviour, so defect detection effectiveness rises with no extra redundancy.
Worked check: With the corrected design a defect such as "quantity <= 100" coded as "quantity < 100" is detected by test 100 (B5), while the original single test (e.g. 50) would pass. A defect that accepts 101 is found by B6, and one that accepts 0 by B1.
<table>
<tr><th>Original defect type</th><th>Found by original design?</th><th>Found by corrected design?</th></tr>
<tr><td>Accepts 0</td><td>No</td><td>Yes (B1)</td></tr>
<tr><td>Rejects 100</td><td>No</td><td>Yes (B5)</td></tr>
<tr><td>Accepts 101</td><td>No</td><td>Yes (B6)</td></tr>
<tr><td>Accepts text</td><td>No</td><td>Yes (EC4)</td></tr>
<tr><td>Crashes on blank</td><td>No</td><td>Yes (EC6)</td></tr>
</table>
Detailed explanation: The original design placed values such as -5, 0, 50, 100 and 150 into one class called "quantity". Because that class contains both accepted and rejected values, a test with 50 passes and the tester assumes the rest of the class is fine. The corrected design follows three rules: (1) a class is either entirely valid or entirely invalid; (2) the classes together cover the whole input domain (numbers, text, blank, decimals); (3) boundaries are tested separately because they are where comparison operators are written wrongly.
</td><td>3</td></tr>
</table>

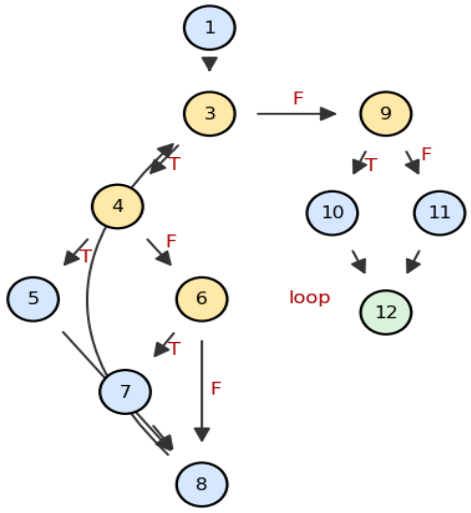
Field	Valid class	Invalid classes	Boundary tests
Quantity	1-100	<1, >100, text, blank, decimal	0,1,2,99,100,101
Coupon code	6-10 alphanumerics	<6, >10, symbols, expired, used	5,6,7,9,10,11 characters
Price filter	0-99999	negative, >99999	-1,0,1,99998,99999,100000
Pin code	6 digits	5 or 7 digits, letters	5,6,7 digits

Effectiveness: Estimated defect detection for typical input-validation faults rises from roughly one in three (single mixed class) to nearly all with the corrected design, while the number of tests grows only from 3 to 13.

Conclusion: Separating classes and adding boundaries converts a vague test list into a design that detects every common input-validation defect with a small number of tests.

2	**Concept recap:** Test adequacy means showing, with a stated criterion, that the important behaviours were exercised. Number of executed tests is not an adequacy criterion. **Errors in strategy and coverage assumptions** • Random testing was the only technique; no equivalence classes, boundaries or limits were derived from requirements. • "High coverage" was claimed from the number of tests executed, not from any coverage measure (requirements, partitions, boundaries, code). • No expected results (test oracle), no traceability, results not repeatable. **Root causes** • Probability that random data hits an exact limit (e.g. 50,000) is tiny, so boundary defects stay hidden. • Input domain was never modelled (limits per transaction, daily limit, balance, account status). • Volume of tests was confused with quality; no adequacy criteria were defined. **Correct black-box techniques** • **ECP:** below-min, valid, above-max, insufficient balance, invalid format. • **BVA:** for limit 100-50,000 test 99, 100, 101, 49,999, 50,000, 50,001. • **Decision table:** balance OK / limit OK / account active combinations. • **State transition:** account Active -> Locked -> Closed behaviour. **Error guessing** only as a supplement. **Justification:** These techniques are derived from the specification, so every rule and limit is covered by a named test (traceable, repeatable, measurable). Adequacy can be shown as 100% partitions + 100% boundaries + 100% decision rules, which gives reliable evidence instead of a guess. **Probability argument:** If the valid range is 100 to 50,000 and random values are drawn from 0 to 1,000,000, the chance that one random value equals exactly 100 or 50,000 is about 2 in 1,000,000. Boundary defects are therefore almost never found by random testing. **Missed defect** | **Technique that finds it** Limit 50,000 coded as < instead of <= | BVA (50,000) Daily limit not accumulated | Decision table / ECP on daily total Locked account can transact | State transition testing Balance equal to amount rejected | BVA on balance **Detailed explanation:** The QA engineer's statement "high coverage" has no definition. Coverage must be expressed against a model: requirement coverage (every requirement has a test), partition coverage (every class has a test), boundary coverage (every limit has tests on both sides) and code coverage. None of these was measured. The strategy error is therefore twofold: the technique is unstructured and the evidence of adequacy is invalid. **Criterion** | **Banking example** | **Measure** Requirement | Transfer, deposit, withdraw, limit, statement | tests per requirement Partition | valid, below min, above max, over balance | classes with a test Boundary | min-1, min, max, max+1 | limits tested Decision | limit OK x balance OK x account active = 8 rules | rules executed State | Active, Locked, Closed | transitions executed **Reliability argument:** Structured tests can be re-run after every change (regression) with the same inputs and expected results; random tests give different data each time and therefore cannot prove that a fixed defect stays fixed. **Conclusion:** Random testing may remain as an additional exploratory activity, but the main strategy must be structured black-box design with measurable criteria.	3

3	**Concept recap:** A CFG has one node per basic block or decision. Decision nodes have two outgoing edges (true/false); loops have a back edge; there should be one exit node. **Sample module used:** 1 read n; s=0; i=1 | 3 while(i<=n) | 4 if(a[i]>0) | 5 s+=a[i] | 6 else if(a[i]<0) | 7 neg++ | 8 i++ | 9 if(s>100) | 10 print High | 11 print Normal | 12 exit **Errors in the CFG and path testing** • The while decision (node 3) and its back edge 8 -> 3 were missing, so the loop was drawn as straight-line code. • The else-if decision (node 6) and the false edges (4->6, 6->8, 3->9) were missing. • No single join/exit node, so branches never merged; paths were counted by guess, not by V(G). **Root causes** • Flowchart thinking: statements drawn, not decisions. • Implicit else and loop exits ignored. • No cross-check using cyclomatic complexity; no review or tool support.	5



Correct CFG: Nodes N = 11, edges E = 14, predicate nodes = 4 (3, 4, 6, 9). **V(G) = E - N + 2 = 14 - 11 + 2 = 5 = P + 1**, so 5 independent paths are needed.

Path	Route	Test data	Expected
P1	1-3-9-11-12	n=0	Normal (loop skipped)
P2	1-3-4-5-8-3-9-11-12	a=[5]	Normal
P3	1-3-4-5-8-3-9-10-12	a=[150]	High
P4	1-3-4-6-7-8-3-9-11-12	a=[-3]	neg=1, Normal
P5	1-3-4-6-8-3-9-11-12	a=[0]	Normal

Justification: The basis set executes every edge at least once, including loop entry, exit and each else branch, and V(G)=5 is the minimum number of tests that guarantees it. Defects in loop handling, sign handling and the threshold are therefore exposed.

Why the original path testing failed: With the loop and else nodes missing, only about 2 paths were identified instead of 5. Tests such as n=0 (loop skipped), a zero element and a negative element were never designed, so loop-skip and sign-handling defects stayed hidden.

Edge	Covered by
1-3	P1-P5
3-4 / 3-9	P2 / P1
4-5 / 4-6	P2,P3 / P4,P5
6-7 / 6-8	P4 / P5
8-3 (back edge)	P2-P5
9-10 / 9-11	P3 / P1,P2,P4,P5

Step-by-step construction of the CFG:

Step	Action	Result
1	Number statements and mark predicates	predicates: while, if, else-if, if
2	Create node per block, join node for i++	11 nodes
3	Add two edges (T/F) for every predicate	8 edges
4	Add sequential edges and the loop back edge	6 edges
5	Count E, N and verify V(G) = P + 1	14, 11, 5 = 4 + 1

Detailed explanation: Missing decision nodes means the tester believed there was one path through the code. Because the while loop was drawn as sequential flow, no test with zero iterations or multiple iterations was designed. Missing the else-if node removed the zero/negative element paths. Missing the join node made it unclear where the three branches of the loop body meet, which hides the back edge.

Defects each path detects: P1: wrong handling of empty input; P2: wrong accumulation; P3: wrong threshold (> versus >=); P4: negative counter error; P5: zero element wrongly counted.

Conclusion: Every one of the 14 edges is covered by the five basis paths, which proves branch adequacy and gives a countable, reviewable test set.

4

Concept recap: Statement coverage = executed statements / total statements. It is the weakest white-box criterion.

Example: if (age >= 18 && income > 5000) approve(); A single test (20, 6000) gives 100% statement coverage.

Gaps in coverage criteria

- False outcome of every decision was never executed (implicit else untested).
- Individual conditions were never made false, so errors such as > for >= or || for && are not exposed.
- Statement coverage only proves a line ran, not that its condition was evaluated correctly.

Why statement coverage alone is insufficient: It is the weakest criterion: it ignores branches, conditions and combinations; a module can reach 100% while half the decision outcomes are untested.

Recommended strategy

- **Branch (decision) coverage:** add (17, 6000) so the decision is false as well as true.
- **Condition coverage / MC/DC:** (20,6000)->T, (17,6000)->F, (20,4000)->F show each condition independently affects the outcome.
- **Basis path coverage** for modules with loops or nested decisions.

Justification: Subsumption: Path > MC/DC > Condition/Decision > Branch > Statement. Moving to branch and MC/DC finds the logical errors that statement coverage hid, with only n+1 tests for n conditions.

Second example (loop): for(i=0;i<n;i++) if(a[i]<0) neg++; A test with one negative element gives 100% statement coverage but never tests n=0, no negative elements, or loop exit.

Coverage achieved	Percentage for single test (20,6000)
Statement	100%
Branch	50% (false branch missing)
Condition	50% (each condition never false)
MC/DC	33%

Detailed explanation: A test that makes a decision true executes the body and therefore all its statements. The false outcome of the same decision executes no statement, so statement coverage never needs it. Logical errors such as replacing && with ||, or > with >=, can only be seen when a test makes the decision take the other outcome or flips a single condition.

Fault injected	Caught by statement coverage test (20,6000)?	Caught by branch/MC/DC tests?
&& changed to ||	No	Yes: (17,6000) approves wrongly
> changed to >=	No	Yes: income 5000
age >= 18 changed to > 18	No	Yes: age 18
Condition removed	No	Yes: (20,4000)

Strategy for the module: 1) measure branch coverage with a tool; 2) list uncovered branches; 3) add tests until 100%; 4) apply MC/DC to compound conditions in critical code; 5) use basis paths where loops exist. Expected outcome: logical defects found in unit testing rather than in production.

Conclusion: Statement coverage should be a minimum entry criterion only. Release criteria should require branch coverage and, for critical logic, MC/DC.

5

5

Concept recap: For a range 0-100 the standard partition is one valid class and two invalid classes (below and above), plus type-based invalid classes.

Errors in the partitioning

- Only two classes by sign; "positive" mixes valid ages 1-100 with invalid ages above 100.
- Zero is unclassified, although 0 is a valid age in the 0-100 range.
- No invalid classes for non-numeric, decimal or empty input; no upper-limit class.
- BVA absent: -1, 0, 100, 101 not tested.

Root cause: Classes were made from a mathematical property (sign) instead of the specification rule (0 <= age <= 100), so the valid-vs-invalid limits were never modelled.

Corrected partitions and boundaries: Valid vs invalid classes are now disjoint:

Class	Type	Sample inputs	Expected
EC1: age < 0	Invalid	-1, -50	Reject
EC2: 0 <= age <= 100	Valid	0, 50, 100	Accept
EC3: age > 100	Invalid	101, 150	Reject
EC4: non-numeric	Invalid	abc, @#	Reject
EC5: decimal	Invalid	25.5	Reject
EC6: empty	Invalid	(blank)	Reject
BVA	Boundaries	-1, 0, 1, 99, 100, 101	R, A, A, A, A, R

Justification: Every valid/invalid limit and data-type failure is covered by at least one class, and BVA checks the exact edges, so input validation errors are caught at the field level.

Defects that the old design misses: Age = 101 accepted (it was in the positive class), age = 0 treated as unclassified, text input crashing. The new design detects all of them.

Test	Input	Expected	Class
1	-1	Reject	EC1
2	0	Accept	EC2 boundary
3	100	Accept	EC2 boundary
4	101	Reject	EC3 boundary
5	abc	Reject	EC4
6	(blank)	Reject	EC6

Detailed explanation: Dividing by sign is a mathematical convenience but it is not what the specification says. The specification defines valid ages as 0 to 100 inclusive. A correct partition therefore needs the numbers 0 and 100 as part of the valid class and the values below 0 and above 100 as two separate invalid classes. Because the original design had no class above 100, a system that accepted 150 would never be detected.

Original class	Problem	Corrected class
Negative numbers	Fine, but only one invalid class	Age < 0
Positive numbers	Contains valid 1-100 and invalid >100; 0 missing	0-100 valid; >100 invalid
(none)	Non-numeric, decimal, blank unclassified	EC4, EC5, EC6

Robustness: Robust input validation means every invalid class gets a clear error message and the stored value is never changed. The new tests verify both.

Conclusion: Six classes plus six boundary points validate the age field completely with about twelve tests.

3

6	**Concept recap:** Condition coverage looks at atomic conditions; decision coverage looks at the whole boolean expression; they are different criteria. **Note:** The question text is incomplete in the paper (it stops at "if(A && B"). It is answered for the decision if (A && B). **Error in the claim:** Condition coverage only requires each condition (A, B) to be True and False once. Tests (A=T,B=F) and (A=F,B=T) satisfy it, yet the decision is always False, so the True branch is never executed: condition coverage does NOT imply decision coverage. **Root cause:** Confusing condition coverage with decision coverage; short-circuit evaluation also hides B when A is False.	-

Test	A	B	Decision
T1	T	T	True
T2	T	F	False
T3	F	T	False

Correction: T1-T3 give Condition/Decision coverage and MC/DC: A is shown independent by T1 vs T3, B by T1 vs T2 (n+1 = 3 tests). This detects && written as || and missing conditions.

Why a short-circuit matters: In (A && B), when A is False, B is never evaluated. A test with A False therefore cannot prove anything about B. MC/DC chooses pairs where only one condition changes and the decision outcome changes.

Criterion	Tests	Decision both T and F?
Condition (T,F),(F,T)	2	No (always False)
Condition (T,T),(F,F)	2	Yes, but B not independently shown
Condition/Decision	(T,T),(F,F)	Yes
MC/DC	(T,T),(T,F),(F,T)	Yes, each condition independent

Detailed explanation: Consider the faulty code if (A || B). Tests (T,F) and (F,T) still satisfy condition coverage and still produce True for both, so the fault is invisible. With MC/DC the test (T,F) is required to give False for && and the faulty || gives True, so the fault is revealed.

Test	A	B	Correct (A&&B)	Faulty (A||B)	Detected?
T1	T	T	T	T	No
T2	T	F	F	T	Yes
T3	F	T	F	T	Yes

Conclusion: The claim must be restated as: condition coverage and decision coverage achieved using three tests (MC/DC).

Conclusion: The developer's claim of full condition coverage is acceptable only if decision coverage is also shown; the correct target is MC/DC.

7

Concept recap: A decision table lists every combination of conditions; with n binary conditions there are 2^n rules, which can be merged using don't-care (-) entries.

Assumption: Conditions: Premium Member (Y/N), Coupon Applied (Y/N), Coupon Valid (Y/N, relevant only if applied). Premium = 10%, coupon = 5%, they stack.

Errors in table construction

- Only the combinations Y/Y and N/N were written; mixed combinations (Premium without coupon, coupon without premium) were skipped.
- Coupon validity (valid/expired) was not a condition, so the error action was missing.
- No don't-care handling, no default rule, and actions were not tied to each rule.

Root cause: The table was built from typical cases instead of enumerating 2^n combinations systematically.

Rule	Premium	Coupon applied	Coupon valid	Action (discount)
R1	Y	Y	Y	15% (10+5)
R2	Y	Y	N	10% + invalid-coupon message
R3	Y	N	-	10%
R4	N	Y	Y	5%
R5	N	Y	N	0% + invalid-coupon message
R6	N	N	-	0%

Justification: All 2^3 = 8 combinations are covered (R3 and R6 merge don't-cares), one test per rule. Price calculation errors in the previously missed rules R1, R2, R4, R5 are exposed before release.

Sample price calculation: Price 1000: R1 gives 1000 x 0.85 = 850; R2 gives 900 and an error message; R4 gives 950; R3 gives 900; R5 and R6 give 1000. The defect "discount added twice" appears only in R1.

Rule	Price	Expected final price
R1	1000	850
R2	1000	900 + message
R3	1000	900
R4	1000	950
R5	1000	1000 + message
R6	1000	1000

Detailed explanation: Decision tables are built in four steps: list the conditions, calculate 2^n combinations, define the action for each combination, then merge rules that differ in only one irrelevant condition. The original table was built from examples a tester remembered, so Premium without a coupon and coupon without Premium were not considered.

Step	Result for the discount system
Conditions	Premium, Coupon applied, Coupon valid
Combinations	2^3 = 8
Merge	Coupon valid is irrelevant when no coupon: 8 -> 6 rules
Tests	6 (one per rule), plus price boundary 0 and maximum

Production risk avoided: Wrong total price, wrong tax base, missing error message for expired coupons, customer complaints and refunds.

Conclusion: A complete decision table links every condition combination to an action and gives one test per rule, so production price errors are prevented.

3

8

Concept recap: Loop testing is a white-box technique that targets the loop control variable and the number of iterations.

Errors in the loop strategy: Only n=1 and n=max were tested. Missing: n=0 (loop skipped), n=2, typical value, n=max-1 and n=max+1 (loop bound exceeded).

Root causes

- Assumption that a loop always executes at least once.
- Off-by-one risk (<= vs <) ignored; the bound was never treated as a boundary value.
- Loop treated like straight-line code, no negative tests.

Simple loop (max = m): Test n = 0, 1, 2, typical, m-1, m, m+1 (m+1 must be rejected/handled).

Nested loops

- Start with innermost loop; set outer loops to their minimum.
- Test innermost: min, min+1, typical, max-1, max (and one below/above).
- Move outward, keeping inner loops at typical values; finally all loops at min, then all at max.
- Concatenated loops: test independently if no shared data; otherwise treat as nested.

Justification: Each iteration count exercises a different path (skip, enter, repeat, exit, overflow), so off-by-one errors, uninitialised variables, infinite loops and boundary overflow are detected and path integrity of the back edge and exit edge is proven.

Example: for(i=1;i<=max;i++) process(i). If the programmer wrote i<max the last item is never processed: found only by n=max. If the loop is entered with n=0 and a variable is uninitialised: found only by n=0.

Loop type	Tests	Defect found
Simple	0	Skip error, uninitialised data
Simple	1, 2	Entry/exit and accumulation errors
Simple	m-1, m	Off-by-one at upper limit
Simple	m+1	Overflow, missing bound check
Nested	inner min/typical/max, outer typical	Interaction and reset errors

Detailed explanation: A loop has four distinct behaviours: not entered, entered once, repeated, and exited at the limit. Testing only n=1 and n=max exercises entry and exit but never the case where the controlling condition is false at the start, nor the case where the counter goes one past the limit. Those two cases are the origin of the "array index out of bounds" and "never initialised" defects.

n value	Behaviour tested	Expected
0	Loop not entered	Result for empty data
1	Single iteration	Correct first result
2	Repeated iteration	Accumulation correct
max-1	Near limit	Correct
max	At limit	Correct, loop ends
max+1	Beyond limit	Error or ignored, no crash

Nested loop example: Outer loop i=1..3, inner loop j=1..4: test inner j=0,1,2,4,5 with i=1; then outer i=0,1,3,4 with j typical (2); finally both at max.

Conclusion: Following the full loop test set proves that the entry edge, back edge and exit edge of every loop are executed.

5

9	**Concept recap:** Error guessing uses experience to guess where defects are likely. It is useful, but it is not systematic and cannot demonstrate completeness. **Errors in using only error guessing** • Depends on tester experience; no systematic or measurable coverage. • Not repeatable or traceable to safety requirements; no adequacy criterion. • Misses combinations, boundaries and event sequences (state-dependent behaviour). **Impact on test adequacy:** Unknown residual risk, undetected failures at limits or in fault states, and failure to meet safety standards that require structured coverage. **Corrected strategy** • **BVA:** pedal position 0-100%: -1, 0, 1, 99, 100, 101; wheel speed 0 and max; response time at limit +/-1 ms. • **State Transition:** Idle -> Braking -> ABS-Active -> Braking -> Idle; any -> Fail-safe on sensor fault. • Test every valid transition and illegal ones (e.g. Idle + wheel-lock event, Fail-safe + pedal release must stay in Fail-safe). • Keep error guessing only as a supplement (sensor dropout, stuck pedal). **Justification:** BVA covers every numeric limit, state testing covers every sequence and invalid event, and both trace to requirements, so coverage is measurable and complete; this gives the higher safety assurance a braking system needs. **State model:** Idle -> Braking (pedal pressed) -> ABS-Active (wheel lock) -> Braking (slip normal) -> Idle (pedal released); any state -> Fail-safe (sensor fault). **Event in state** | **Expected result** Pedal pressed in Idle | Braking Wheel lock in Braking | ABS-Active Wheel lock in Idle | Ignored (illegal) Pedal released in Fail-safe | Stay Fail-safe Sensor fault in ABS-Active | Fail-safe, warning lamp **Detailed explanation:** A braking system must be shown to behave correctly for every valid input and every valid and invalid sequence of events. Error guessing finds only what the tester imagines. Safety standards (for example ISO 26262) require systematic techniques with evidence of coverage, so using error guessing alone is both a technical and a compliance error. **Parameter** | **BVA values** | **Expected** Pedal position (%) | -1,0,1,50,99,100,101 | Reject, no brake, light brake, ..., full, reject Wheel speed (km/h) | 0,1,max-1,max,max+1 | No ABS at 0; limit handling Response time (ms) | limit-1, limit, limit+1 | Pass, pass, fail flagged **State transition coverage:** 0-switch covers every transition once (6 valid + 3 invalid); 1-switch covers pairs such as Braking -> ABS -> Braking. Error guessing adds sensor noise and stuck-pedal cases as extras. **Conclusion:** Error guessing remains useful to add extra safety cases, but BVA and state testing provide the systematic, auditable coverage required for a safety-critical system.	5

10

Concept recap: An if without else still has two edges: the true edge and an implicit false edge that skips the body. Branch coverage needs both.

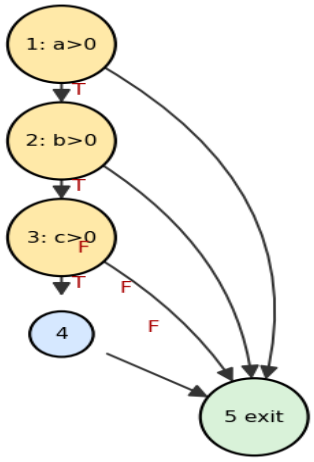
Sample code: if (a>0){ if (b>0){ if (c>0){ process(); } } } return; (no else branches)

Errors in graph construction

- The implicit else (false) edges 1->5, 2->5, 3->5 were not drawn, so the graph had a single route.
- One test (a,b,c > 0) executes every statement (100% statement) but none of the three false edges (0% on error paths).

Root causes

- Graph drawn from statements; an if without else was assumed to have no alternative flow.
- Missing join node and no V(G) check: N=5, E=7 gives V(G)=4, a 4-path graph was expected.



Corrected independent paths (V(G) = 7 - 5 + 2 = 4): (T = true, F = false edge)

Path	Route	Test data
P1	1-2-3-4-5	a>0, b>0, c>0
P2	1-5	a<=0
P3	1-2-5	a>0, b<=0
P4	1-2-3-5	a>0, b>0, c<=0

Result: All 6 branches (3 true + 3 false) are executed, giving 100% branch adequacy.

Coverage before and after: Original graph (no else edges): 1 path, 4 edges counted. Corrected graph: 7 edges, 4 paths.

Metric	Original	Corrected
Edges	4	7
Branch coverage with 1 test	not measurable (0% of else edges)	4/7 = 57%
Tests for 100% branch	1	4 (P1-P4)

Detailed explanation: In a CFG every decision must have two outgoing edges even when the source has no else clause. If the false edge is omitted, the graph claims only one route exists, and the tester concludes that a single test is enough. The statement count is satisfied but 3 of 6 decision outcomes (the three false edges) are never executed.

Edge	Type	Test that executes it
1->2	true	P1
1->5	false (implicit else)	P2
2->3	true	P1
2->5	false	P3
3->4	true	P1
3->5	false	P4

5

	Branch adequacy: Four tests give 6/6 decision outcomes = 100% branch coverage, and V(G) = 4 confirms that no independent path is missing. **Conclusion:** Drawing the implicit else edges, checking V(G) = 4 and executing P1-P4 gives complete branch adequacy.	

4. Summary of Errors, Causes and Corrections

Q	Main error	Root cause	Correction
1	Mixed valid/invalid classes; no BVA	Classes not disjoint; spec ignored	Separate classes + 0,1,100,101 boundaries
2	Random testing claimed as coverage	No adequacy criteria; limits not modelled	ECP, BVA, decision table, state testing
3	CFG without loop and else nodes	Flowchart thinking; no V(G) check	Full CFG, V(G)=5, five basis paths
4	100% statement, logic errors missed	Weakest criterion used	Branch, condition, MC/DC, path
5	Only negative/positive classes	Sign used instead of range rule	Six classes + BVA
6	Condition coverage taken as enough	Confused with decision coverage	Condition/decision + MC/DC (3 tests)
7	Missing Premium/Coupon combinations	Rules not enumerated 2^n	Exhaustive 6-rule table
8	Only n=1 and n=max	Loop assumed to run	0,1,2,m-1,m,m+1 and nested method
9	Error guessing only	Ad-hoc, no measure	BVA + state transition + guessing
10	Else branches missing in CFG	Implicit else ignored	CFG with 7 edges, 4 paths

5. Overall Conclusion

All ten scenarios share one lesson: a test design is only as good as the model behind it. When the model (partitions, graph or table) is incomplete, every test derived from it is incomplete, and a reported coverage figure gives false confidence. Correct models are built from the specification and the code structure, checked with simple formulas (such as V(G) and 2^n rules), and judged against explicit adequacy criteria. Applying these corrections makes tests traceable, repeatable and measurable, and detects boundary, logical and loop defects that the original approaches missed.

6. Preventive Checklist for Testers

1. Derive classes from requirements; each class must be disjoint and homogeneous.
2. Always add boundary values and invalid classes (below, above, wrong type, empty).
3. Draw the CFG from decisions, not from statements; include else, loop back edge and exit; verify with V(G) = E - N + 2.
4. Choose a coverage criterion before testing and report it by name (branch, MC/DC, path).
5. Use decision tables for combinations and state models for sequences.
6. Use random and error-guessing tests only as a supplement.
7. Review test design with a second person and trace each test to a requirement.

Appendix A: Consolidated Corrected Test Suite

ID	Question	Input / condition	Expected	Technique
T01	Q1	quantity 0	Reject	BVA
T02	Q1	quantity 1	Accept	BVA
T03	Q1	quantity 100	Accept	BVA
T04	Q1	quantity 101	Reject	BVA
T05	Q1	quantity abc	Reject	ECP
T06	Q2	transfer 99 / 100	Reject / Accept	BVA
T07	Q2	transfer 50,000 / 50,001	Accept / Reject	BVA
T08	Q3	n=0	Normal	basis path P1
T09	Q3	a=[150]	High	basis path P3
T10	Q4	(17,6000)	Not approved	branch
T11	Q4	(20,4000)	Not approved	MC/DC
T12	Q5	age -1	Reject	BVA
T13	Q5	age 0 / 100	Accept	BVA
T14	Q5	age 101	Reject	BVA
T15	Q6	A=T,B=T / T,F / F,T	T / F / F	MC/DC
T16	Q7	Premium=Y, coupon valid	15% discount	decision table
T17	Q7	Premium=N, coupon expired	0% + message	decision table
T18	Q8	n=0 / 1 / 2 / max-1 / max / max+1	per loop plan	loop testing
T19	Q9	pedal -1 / 0 / 100 / 101	Reject/ok/ok/Reject	BVA
T20	Q9	wheel lock in Idle	Ignored	state
T21	Q10	a>0,b>0,c>0	Process	path P1
T22	Q10	a<=0	Skip	path P2
T23	Q10	a>0,b<=0	Skip	path P3
T24	Q10	a>0,b>0,c<=0	Skip	path P4

Appendix B: Cyclomatic Complexity Worked Computations

Question	Nodes N	Edges E	Predicates P	V(G) = E - N + 2	V(G) = P + 1
Q3 (loop with if/else-if)	11	14	4	5	5
Q10 (three nested ifs)	5	7	3	4	4
Q10 (corrected full else version)	8	10	3	4	4

Appendix C: Glossary

Term	Meaning
Error	Human mistake in design or code
Defect	Flaw in the product caused by an error
Failure	Observable wrong behaviour at run time
Test adequacy	Degree to which a criterion is satisfied by a test suite
Oracle	Source of the expected result
Regression test	Re-test after change to check nothing else broke
Infeasible path	Path that no input can execute
Short-circuit evaluation	Skipping the right operand when the left decides the result

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand